

Nom:

Prénom:

BACCALAURÉAT GÉNÉRAL

ÉPREUVE D'ENSEIGNEMENT DE SPÉCIALITÉ

Session 2026

NUMERIQUE ET SCIENCES INFORMATIQUES

DURÉE DE L'ÉPREUVE : 2 heures

*Ce sujet comporte 4 pages numérotées de 1 à 4
La dernière page comporte une annexe que vous pouvez détacher.
Le candidat doit traiter les deux exercices proposés.*

L'utilisation d'une calculatrice n'est pas autorisée

Tournez la page S.V.P.

Exercice 1 ►

PARTIE A

- Donner l'ordre de parcours des sommets du graphe.
- Donner la table de routage de F à l'initialisation du protocole RIP.

ABCEDF

Routeur	Nombre de sauts	Prochain routeur
D	1	–

- Donner la table de routage de A après une première itération de RIP (deux réponses sont possibles).

Routeur	Nombre de sauts	Prochain routeur
B	1	–
C	1	–
E	1	–
D	2	E ou C

- Donner le numéro de l'itération de RIP à partir duquel les tables des routeurs du réseau ne varient plus.

Les tables de routage ne varient plus à partir de la troisième itération de RIP.

- Donner la nouvelle table de routage de A après stabilisation de RIP (deux réponses sont possibles).

Routeur	Nombre de sauts	Prochain routeur
B	1	–
C	1	–
E	1	–
D	2	E ou C
F	3	E ou C

PARTIE B

- Compléter le code de la méthode `relie`.

```

</> Code Python
1 def relie(self, autre):
2     if autre not in self.voisins:
3         self.voisins.append(autre)
4         self.nb_sauts[autre] = 1
5         self.prochain[autre] = None
6         if self not in autre.voisins:
7             autre.relie(self)

```

- Écrire la méthode `relie_liste` de la classe `Routeur` qui prend en paramètre une liste de routeurs `lst` et qui relie le routeur `self` à chacun des routeurs de la liste `lst`.

```

</> Code Python
1 def relie_liste(self, lst):
2     for autre in lst:
3         self.relie(autre)

```

8. Écrire les instructions manquantes pour relier les routeurs de manière à obtenir le graphe de la figure ??.

</> Code Python

```
1 A.relie_liste([B, C, E])
2 D.relie_liste([C, E, F])
3 C.relie(E)
```

9. Compléter le code de la méthode `met_a_jour_table` ci-dessus.

</> Code Python

```
1 def met_a_jour_table(self, autre):
2     for r in autre.nb_sauts:
3         if r != self:
4             if (r not in self.nb_sauts or self.nb_sauts[r] > \
5                 autre.nb_sauts[r] + 1):
6                 self.nb_sauts[r] = autre.nb_sauts[r] + 1
7                 self.prochain[r] = autre.nom
```

10. Écrire la méthode `itere_rip` qui prend en paramètre le routeur `self` et met à jour sa table de routage lorsqu'il reçoit la table de routage de chacun des routeurs présents dans la liste de ses voisins.

</> Code Python

```
1 def itere_rip(self):
2     for autre in self.voisins:
3         self.met_a_jour_table(autre)
```

11. Écrire une fonction qui prend en paramètre une liste de routeurs `l_routeurs` et qui réalise une itération du protocole RIP pour tous les routeurs de `l_routeurs`.

</> Code Python

```
1 def iteration_rip(l_routeurs):
2     for r in l_routeurs:
3         r.itere_rip()
```

12. Écrire une version modifiée du code de la méthode `itere_rip` de la classe `Routeur` de telle sorte que celle-ci renvoie `True` dans le cas où le routeur `self` a procédé à une modification de sa table de routage au cours de l'exécution de la méthode `itere_rip`, et `False` sinon.

</> Code Python

```
1 def itere_rip(self):
2     modifie = False
3     for autre in self.voisins:
4         if self.met_a_jour_table(autre):
5             modifie = True
6     return modifie
```

13. Compléter le code du programme principal afin que celui-ci mette à jour les tables de routage des routeurs présents dans la liste `liste_routeurs` jusqu'à ce qu'il ne soit plus nécessaire de faire des mises à jour des tables de routage.

</> Code Python

```
1 modifie = True
2 while modifie:
3     modifie = False
4     for r in liste_routeurs:
5         if r.itere_rip():
6             modifie = True
```

Exercice 2 ►

PARTIE A

14. Justifier que l'attribut `Num_Objet` peut être choisi comme clé primaire de la table `Gaia`.

L'attribut `Num_Objet` peut être choisi comme clé primaire de la table `Gaia` car il identifie de manière unique chaque objet cartographié. En effet, chaque objet a un numéro d'identification unique, ce qui garantit que les valeurs de `Num_Objet` sont distinctes pour chaque enregistrement de la table. De plus, cet attribut est stable et ne change pas au fil du temps, ce qui en fait un bon candidat pour une clé primaire.

15. Proposer le schéma relationnel de la table `Type` en soulignant la clé primaire.

`Type (Type_Objet : String, Libelle : String)` où `Type_Objet` est la clé primaire de la table.

PARTIE B

16. Expliquer pourquoi le code SQL suivant ne fonctionne pas :

```
INSERT INTO Type VALUES ('BD', 'Trou Noir');
```

Cette requête échoue à cause d'une violation de contrainte d'unicité (ou d'intégrité de clé primaire). La valeur 'BD' existe déjà dans la table `Type` (associée à 'Naine Brune'). Comme `Type_Objet` est la clé primaire, on ne peut pas insérer deux lignes avec la même valeur pour ce champ.

17. Indiquer le résultat de la requête suivante exécutée sur l'extrait présenté :

```
SELECT Nom_Objet, Parallaxe FROM Gaia WHERE Type_Objet = 'Planet';
```

Nom Objet	Parallaxe
Proxima Cen b	768,067
Lalande 21185 b	392,753

18. Écrire la requête qui permet de récupérer le nom du système, le nom de l'objet et le libellé du type pour des objets ayant une parallaxe supérieure à 400 millisecondes d'arc et étant des 'Étoile'.

```
SELECT Gaia.Nom_Systeme, Gaia.Nom_Objet, Type.Libelle_Objet
FROM Gaia
JOIN Type ON Gaia.Type_Objet = Type.Type_Objet
WHERE Gaia.Parallaxe > 400 AND Type.Type_Objet = 'Etoile';
```

19. Le chercheur a besoin du nom, du libellé de leur type, de la déclinaison et de la parallaxe de tous les objets du système stellaire Alpha du Centaure ('alf Cen'). Écrire la requête qui permet d'extraire ces informations de la base de données.

```
SELECT Gaia.Nom_Objet, Type.Libelle_Objet, Gaia.Declinaison, Gaia.Parallaxe
FROM Gaia
JOIN Type ON Gaia.Type_Objet = Type.Type_Objet
WHERE Gaia.Nom_Systeme = 'alf Cen';
```